# Agentic Sprint Planner

Updated System Architecture & Design Document

CSE 496 — Kerem Göbekçioğlu

## 1. Product Overview

A web-based AI sprint planning assistant for small software teams (5–15 people). The user configures their team once, pastes their backlog as epics, selects a domain, and receives a full sprint plan — broken into user stories and tasks, estimated in Fibonacci story points, and assigned to developers within real capacity limits.

The system supports two entry modes:

- Fresh Start — no prior history. The system uses conservative defaults and begins building memory from the first sprint review.
- Continuous Mode — existing team with sprint history. The system automatically loads two memory sources (preference profile + sprint reference) and uses them to calibrate every agent.

Before anything is pushed to Trello, the user reviews and edits the plan in a dedicated Review Screen. Every edit is logged and fed back into the preference profile, and every sprint outcome feeds the reference history — making the plan smarter over each cycle.

| Parameter | Value | Notes |
|---|---|---|
| Target Users | Tech leads & PMs, 5–15 person teams | |
| Sprint Length | 2 weeks (configurable) | |
| Domain Support | E-Commerce, Fintech, Generic | Overridable per sprint |
| Output Target | Trello board (Jira planned) | |
| AI Backend | Gemini Flash (primary) + Ollama (dev fallback) | |
| Stack | Spring Boot 3 · PostgreSQL · React | |

## 2. High-Level Architecture

The system is divided into five layers: a React frontend, a Spring Cloud API Gateway, domain microservices, the AI Orchestrator (which runs the four agents), and external integrations.

Request flow depends on whether the team has prior history:

| Flow A — Fresh Start | Flow B — Continuous |
|---|---|
| User configures new team<br>Pastes epics + selects domain<br>↓ No history found<br>Conservative defaults applied (0.80 capacity factor)<br>↓ **AI Orchestrator**<br>↓ Review Screen<br>↓ Sprint Review Form seeds memory | User selects existing team<br>Pastes new epics<br>↓ System loads:<br>• team_profile.md (preference memory)<br>• sprint_history.md (reference memory, last 3)<br>↓ Both injected into agent prompts<br>↓ **AI Orchestrator**<br>↓ Review Screen (with mismatch warnings) |

## 3. The Four Agents

Each agent has a single responsibility. The Orchestrator calls them in sequence and validates the output of each before passing it to the next. If any agent fails, the pipeline halts and returns a structured error — no silent failures.

### Agent 1 — BreakdownAgent

| | BreakdownAgent |
|---|---|
| Role | Converts each epic into user stories, then breaks each user story into concrete development tasks with acceptance criteria. |
| Input | Epic title + description · Domain · Sprint goal · Team roles · Definition of Done · team_profile.md (preference) · sprint_history.md (last 3 sprints, if available) |
| Output | Structured list of user stories, each with 2–4 child tasks. Each task has: title, description, acceptance criterion, role tag (backend / frontend / qa / devops). |

**Pseudocode:**

```
FUNCTION breakdown(epic, context):
    prompt = build_prompt(epic, context.domain, context.sprint_goal,
                          context.definition_of_done,
                          context.preferences,      // from team_profile.md
                          context.sprint_history)   // from sprint_history.md
    raw = call_llm(prompt)
    stories = parse_user_stories(raw)
    FOR EACH story IN stories:
        validate: has title, has 2+ tasks, each task has acceptance_criterion
        IF invalid: retry once with correction prompt
    RETURN stories
```

Note: Epic → User Story → Task (3-level breakdown). Both memory sources fed into prompt.

### Agent 2 — EstimationAgent

| | EstimationAgent |
|---|---|
| Role | Assigns a Fibonacci story point value to each task using domain context and task complexity signals. Validates every value against the allowed set. |
| Input | Task list from BreakdownAgent · Domain · Sprint goal · velocity_history from sprint_history.md (actual vs planned per task type) |
| Output | Same task list with story_points field added. Every value is in {1, 2, 3, 5, 8, 13}. Tasks with 8 or 13 points are flagged for PM review. |

**Pseudocode:**

```
FUNCTION estimate(tasks, context):
    FOR EACH task IN tasks:
        prompt = build_estimation_prompt(task, context.domain,
                                         context.velocity_history,
                                         context.estimation_errors)  // from
sprint_history.md
        raw_points = call_llm(prompt)
        points = round_to_fibonacci(raw_points)
        IF points NOT IN {1,2,3,5,8,13}: points = nearest_fibonacci(raw_points)
        IF points >= 8: flag_for_review(task)
        task.story_points = points
    RETURN tasks
```

Note: Large tasks (8, 13 pts) are surfaced on the Review Screen as “consider splitting.” Estimation errors from past sprints are also injected, letting the agent self-correct known biases.

## Agent 3 — CapacityAgent

| | CapacityAgent |
|---|---|
| Role | Pure algorithm — no LLM call. Computes each developer’s real available story point budget for this sprint. |
| Input | Developer list (name, role, base_velocity) · Sprint length in days · days_off per developer · Historical completion rate from sprint_history.md |
| Output | Capacity map: { developer_id → available_story_points } |

**Pseudocode:**

```
FUNCTION calculate_capacity(developers, sprint_days, history):
    FOR EACH dev IN developers:
        working_days = sprint_days - dev.days_off
        base_capacity = working_days * (dev.base_velocity / sprint_days)
        IF history[dev.id] EXISTS:
            completion_rate = history[dev.id].avg_completion_rate
            adjusted = base_capacity * completion_rate
        ELSE:
            adjusted = base_capacity * 0.80  // conservative default (fresh start)
```

```
        capacity_map[dev.id] = floor(adjusted)
    RETURN capacity_map
```

### Agent 4 — DistributionAgent

| | DistributionAgent |
|---|---|
| Role | Assigns tasks to developers. LLM ranks fit (role match, workload balance, past preference and past performance); algorithm enforces hard capacity constraints. |
| Input | Estimated task list · Capacity map · Developer roles · team_profile.md (preference) · sprint_history.md (per-developer performance by task type) |
| Output | Final sprint plan: each task has assigned_developer, story_points, sprint_id. Unassigned tasks flagged. |

Note: Unassigned tasks are surfaced on the Review Screen. PM can manually assign or defer to next sprint.

## 4. Review Screen

The Review Screen is the most important feature. Nothing is pushed to Trello until the user approves. This single step solves the trust problem: users stay in control, bad AI outputs are caught before they matter, and every edit becomes preference training data.

### Standard actions

- Edit any field inline (title, points, assignee)
- Drag task to different developer
- Delete task or add a custom task
- Mark task as “defer to next sprint”
- Approve → push to Trello

### Mismatch Warnings (Continuous Mode only)

When sprint_history.md shows a consistent gap between what was planned and what was actually delivered, the Review Screen surfaces it explicitly — it does not silently adjust. This is the kind of insight an experienced PM gives in a retrospective. The system gives it automatically.

Examples of surfaced warnings:

| Warning Type | Example Message |
|---|---|
| Velocity mismatch |  Your team planned 50 pts last 3 sprints but completed 35 on average. This plan is 48 pts — consider deferring lower-priority tasks. |
| Estimation drift |  ‘Analytics tracking’ tasks have been underestimated 3 sprints in a row. Suggested: +2 pts on each. |
| Assignment risk |  Ceren is assigned 13 pts this sprint. Her 3-sprint average is 9 pts completed. Consider redistributing 4 pts. |
| Goal miss pattern |  Sprint goal was ‘Partially’ or ‘No’ last 2 sprints. The team may be over-committing on scope. |

The PM can dismiss any warning. Dismissed warnings are logged but do not affect the preference profile.

## 5. Preference Memory (Edit-Based)

Every change the user makes on the Review Screen is logged as a diff. After the session, PreferenceService aggregates diffs into a team_profile.md. This file captures intent — what the team’s PM explicitly chose to override.

### How it works

- User edits task on Review Screen (rename, re-estimate, reassign)
- EditLogger records the diff: { field, original_value, new_value, task_type, domain }
- After approval, PreferenceService aggregates diffs into team profile
- Profile stored as structured JSON and rendered to readable Markdown
- On next sprint: team_profile.md injected into BreakdownAgent, EstimationAgent, DistributionAgent prompts

### Example team_profile.md

```
# Team Preferences — Trendyol Checkout Squad

## Estimation
- Payment-related tasks: user consistently raises to 5+ pts
- Analytics/tracking tasks: user consistently lowers to 1–2 pts

## Assignment
- Backend API tasks: prefer Ahmet or Buse (never Ceren)
- Mobile tasks: always assign to Ege

## Task Breakdown
- Auth epics always need an extra 'write integration tests' task
```

Preference memory is intent. It reflects what the PM decided to do. It does not track whether those decisions led to good outcomes — that is the job of reference memory (Section 7).

## 6. Sprint Review & Velocity Learning

After the sprint ends, the user closes it via a simple form. This data feeds directly into CapacityAgent for the next sprint and is the primary input to the ReferenceBuilder service (Section 7).

| Field | Description |
|---|---|
| Task completion | Done / Not Done toggle per task |
| Sprint goal result | Did the team hit the sprint goal? Yes / Partially / No |
| Blocker note | Optional free-text: what blocked the sprint? |
| Developer actuals | Actual story points completed per developer (auto-calculated from task toggles) |

What happens with this data:

- completion_rate per developer = tasks_done / tasks_assigned (by story points)
- CapacityAgent uses this rate next sprint to adjust individual budgets
- Developers who consistently over/underdeliver get realistic capacity automatically
- Sprint goal result stored — shown in team dashboard over time
- All data passed to ReferenceBuilder → sprint_history.md updated

## 7. Reference Memory (Outcome-Based)

Reference memory is distinct from preference memory. Preference captures intent (what the PM chose). Reference captures reality (what actually happened). Sometimes they conflict — a team may keep planning 50 pts while consistently completing 35. That tension is surfaced to the PM, not silently resolved.

### ReferenceBuilder Service

- Triggered automatically after each Sprint Review Form submission
- Reads sprint outcome JSON and appends to sprint_history.md
- Maintains a rolling 3-sprint window — older sprints are archived but not injected into prompts
- 3-sprint cap is intentional: recent patterns matter more than history; keeps prompt size bounded

### sprint_history.md format

```
# Sprint History — Trendyol Checkout Squad

## Sprint 4 (completed)
**Goal:** Reduce mobile checkout drop-off  |  **Result:** PARTIALLY
**Velocity:** 38 planned / 30 actual

### Estimation accuracy
- Payment tasks: estimated 5 pts, actual effort ~8 pts (underestimated 3x in a row)
- Analytics tasks: estimated 3 pts, actual effort ~2 pts (overestimated
consistently)

### Developer actuals
| Developer | Planned | Completed | Completion Rate |
|-----------|---------|-----------|-----------------|
| Ege       | 11 pts  | 11 pts    | 100%            |
| Ahmet     | 13 pts  | 9 pts     | 69%             |
| Ceren     | 8 pts   | 5 pts     | 63%             |

### Blockers
Third-party payment SDK docs were outdated — caused 3-day delay on Ahmet's tasks.

## Sprint 3 (completed)
...
```

### What agents do with sprint_history.md

| Agent | Uses from sprint_history.md |
|---|---|
| BreakdownAgent | Which epic types historically produced more/fewer tasks than expected |
| EstimationAgent | Which task categories have consistent estimation errors; adjusts prompts accordingly |
| CapacityAgent | Actual completion rates per developer (direct JSON read, not via prompt) |
| DistributionAgent | Per-developer performance by task type and role tag |

Note: CapacityAgent reads sprint outcome JSON directly (structured data). The other three agents receive sprint_history.md as part of their prompt (natural language context).

## 8. Two Entry Flows

The system detects which mode to use automatically based on whether a team record with at least one completed sprint exists in the database.

| | Flow A — Fresh Start | Flow B — Continuous |
|---|---|---|
| Trigger | New team, or existing team with no completed sprint | Existing team with ≥ 1 completed sprint review |
| team_profile.md | Empty — created after first Review Screen session | Loaded and injected into agent prompts |
| sprint_history.md | Not present | Last 3 sprints loaded and injected |
| CapacityAgent | Uses 0.80 conservative default for all developers | Uses actual completion_rate per developer from history |
| Mismatch warnings | Not shown (no baseline to compare against) | Shown when pattern detected over 2+ sprints |
| After sprint review | Data saved → system transitions to Flow B from next sprint | Data appended to rolling 3-sprint window |

The transition from Flow A to Flow B is automatic. After the first Sprint Review Form is submitted, the team has reference data and will receive continuous mode on the next planning session.

## 9. Domain Context System

Domain selection is made once at team setup and can be overridden per sprint. It changes the language and task templates injected into agent prompts.

| Domain | Typical Epic Style | Prompt Adjustments |
|---|---|---|
| E_COMMERCE | "Reduce checkout drop-off on mobile" | Emphasise A/B test tasks, analytics events, mobile-first UI. Conservative QA tasks. |
| FINTECH | "Add IBAN validation to transfer flow" | Strict acceptance criteria, mandatory review steps, compliance notes, conservative estimates. |
| GENERIC | "Improve user onboarding flow" | Balanced breakdown, neutral language, no domain-specific assumptions. |

## 10. Test Scenarios

Three realistic scenarios modeled after actual Turkish tech team profiles. Each has a manually written reference plan. System output is compared against the reference on three dimensions: coverage, estimation sanity, and assignment logic.

### Scenario A — Trendyol-style E-Commerce Squad

| | Details |
|---|---|
| Team | 6 developers (2 frontend, 2 backend, 1 mobile, 1 QA) · 2-week sprint · Domain: E_COMMERCE |
| Epics | Reduce checkout drop-off rate on mobile Add "save for later" to shopping cart Improve seller dashboard load time |
| Reference criteria | Each epic produces 2+ user stories Mobile epic tasks assigned to mobile dev Performance epic includes a profiling/measurement task Total sprint points: 40–55 |

### Scenario B — Fintech Transfer Flow Team

| | Details |
|---|---|
| Team | 4 developers (3 backend, 1 QA) · 2-week sprint · Domain: FINTECH |
| Epics | Add IBAN validation to money transfer flow End-of-day reconciliation report for settlements |
| Reference criteria | Acceptance criteria are strict and testable (not vague) IBAN epic includes an error-handling task Reconciliation epic includes a QA task for edge cases Conservative estimates — fintech tasks are rarely < 3 pts |

### Scenario C — Generic SaaS Onboarding (Stress Test)

| | Details |
|---|---|
| Team | 5 developers (mixed) · 2-week sprint · Domain: GENERIC · Intentionally vague epics |
| Epics (vague) | Improve onboarding Fix the dashboard Make it faster |
| What this tests | Does the system make reasonable assumptions or produce nonsense? Does it flag ambiguous epics to the user? Does it degrade gracefully (usable plan) rather than failing? |

## 11. Success Criteria

| Type | Criterion | How Measured |
|---|---|---|
| System | End-to-end plan for 3 epics completes in < 90 seconds | Automated timer in test suite |
| System | All story points are in {1, 2, 3, 5, 8, 13} | Validator in EstimationAgent |
| System | No developer assigned more points than capacity | DistributionAgent hard constraint |

| Type | Criterion | How Measured |
|---|---|---|
| System | No silent failures — errors always returned structured | Integration test with bad inputs |
| Quality | Scenario A: ≥ 80% task coverage vs. reference plan | Manual comparison checklist |
| Quality | Scenario B: all tasks have testable acceptance criteria | Manual review — must pass 100% |
| Quality | Scenario C: system degrades gracefully on vague epics | Manual review — output must be usable |
| Learning | After 2 sprints, preference profile changes estimation by ≥ 1 case | Compare sprint 1 vs sprint 3 output |
| Learning | After 2 sprints, mismatch warning fires at least once if velocity gap > 20% | Automated check against sprint_history.md |
| Usability | PM with no training configures team + runs first plan in < 5 min | Observed user test |

## 12. Questions for PM Interview

The following questions are designed to validate design decisions, surface unknown requirements, and gather real sprint data for test scenarios. Ask in Turkish; answers will directly inform system design.

### A. Current Process

- Sprint planlamayı şu an nasıl yapıyorsunuz? Toplantı mı, Jira'da mı, ikisi birlikte mi? Ortalama ne kadar sürüyor?
- Hikaye puanı tahmini konusunda takımda en çok nerede zaman kaybediyorsunuz — tartışma mı, eksik bilgi mi, birileri konuyu bilmiyor mu?
- Bir sprint bittiğinde "bu planı neden kaçırdık?" sorusunun cevabı genellikle ne oluyor?
- PM olarak sprint planda en çok hangi kısmı kendiniz düzeltiyorsunuz — görev kırılımı mı, tahminler mi, atamalar mı?

### B. Existing Tools

- Bahsettiğin otomasyon araçları neler? Jira'nın kendi özellikleri mi yoksa üçüncü parti araçlar mı?
- Slack entegrasyonu derken o araçlar tam olarak ne yapıyor — sprint özeti mi atıyor, task güncellemesi mi bildiriyor?
- Bu araçların seni hayal kırıklığına uğrattığı anlar oluyor mu? Ne eksik kalıyor?

### C. Trust & Control

- Bir AI aracı sana “şu görevleri planladım” dese — bunu direkt Jira’ya göndermesini mi istersin, yoksa önce görmek ister misin?
- AI’nın ürettiği bir görevi düzeltmek için ne kadar çaba harcamaya razısın — 1 tıklama mı, 3 tıklama mı?
- Sistemin bir sprint sonrası kullanıcı düzeltmelerinden öğrenmesini anlamlı bulur musun?
- Sistem sana “Bu takım son 3 sprintte 35 puan tamamladı ama sen 50 puan planlıyorsun” dese — bunu görmek ister misin? Yoksa rahatsız edici mi olur?

## D. Real Data Request

- Elinde anonim bir sprint verisi var mı? (Epic isimleri, görev başlıkları, kim ne aldı, ne bitti ne bitmedi — kişisel bilgi olmadan)
- Takımında genellikle kaç developer var ve rolleri neler?
- İki haftada bir sprint mi yapıyorsunuz? Developer başına ortalama kaç story point düşüyor?
- “Bitti” ile “gerçekten bitti” arasında fark oluyor mu?

## E. Edge Cases & Failure Modes

- Sprint ortasında yeni bir acil iş gelirse ne yapıyorsunuz — mevcut planı mı bozuyorsunuz?
- Birinin izne çıkacağını planlama sırasında biliyorsunuz — bunu kapasiteye nasıl yansıtıyorsunuz şu an?
- Bir epik çok muğlak olsa — “dashboard’u iyileştir” gibi — planlamayı nasıl başlatıyorsunuz?